I. PRELIMINARY

A driving scenario can be represented as $S = (S_a, S_m)$, where S_m denotes map tracks and S_a denotes agent tracks.

1) Map tracks S_m*:* Each map element i contains a type m_i, a position $p_{i,1:N}$, and a state $s_{i,1:T}$, and is denoted as $S_m = \{m_i, p_{i,1:N_i}, c_{i,1:T}\}$, where N_i denotes the total number of points in the map element and T denotes the total number of time steps in the scenario. The type m_i includes lane centerlines, road edges, lane lines, and other elements. The position can be represented by polylines, polygons (crosswalks and speed bumps), or points (stop signs), and the state indicates traffic-light control.

2) Agent tracks S_a*:* Each agent contains a type a_i and a state $s_{i,1:T}$, denoted as $S_a = \{a_i, s_{i,1:T}\}$. The complete state of agent i is represented as $s_i = \{c_i, \theta_i, v_i, a_i, d_i\}$, including global position $c_i = [x_i, y_i, z_i] \in \mathbb{R}^{3\times T}$, heading θ_i, longitudinal and lateral velocities $v_i = \left[v_i^x, v_i^y\right] \in \mathbb{R}^{2\times T}$, longitudinal and lateral accelerations $a_i = \left[a_i^x, a_i^y\right] \in \mathbb{R}^{2\times T}$, and dimensions (length, width, and height) $d_i = [l_i, w_i, h_i] \in \mathbb{R}^{3\times T}$.

II. METHODOLOGY

Our goal is to train a demand-guided model $\mathcal{S} = \text{Demand2Scenario}\big(\mathcal{T}, \text{User}(\mathcal{T})\big)$ that, given the complete scenario text $\mathcal{T}$, generates a driving scenario $\mathcal{S}$ matching the text while satisfying user requirements. Here, $\text{User}(\cdot)$ denotes the user demand, including a customized map style and freely placed agents of different types on the selected map via mouse interaction.

A. MLLM-Powered Interpreter

The interpreter $c_a = \text{Interpreter}(\mathcal{T}, \mathcal{I}, p_0)$ converts the user demand into machine-readable agent codes c_a. Its inputs include the complete scenario text $\mathcal{T}$, the initial scene snapshot $\mathcal{I}$, and the initial agent poses p_0.

1) Agent Codes: The agent code c_a^* is a 13-dimensional integer vector that compactly describes the attributes and motion states of all agents in a scenario. It consists of one dimension for type, speed range, and size range, five dimensions for action trend (start/keep/end), and five dimensions for action type. The type, speed range, and size range are directly extracted from the reconstructed agent tracks S_a^*. For actions, we adopt a more fine-grained extraction strategy; details are provided in Supp. A. We first annotate the raw dataset at each time step, thereby assigning a unique action trend and action type to every temporal window (T_w time steps). Priorities are defined according to the rarity of each action type in the real world, with rarer actions assigned higher priority: U-turn > left/right lane change > left/right turn > acceleration/deceleration > constant speed > stop. If all T_w time steps have the same action type, the action trend is set to "keep" and the action type is set to that type. If the T_w time steps do not share the same action type, only the first and last time steps are considered. If the action type at the first time step has higher priority than that at the last time step, the trend is set to "end" and the action type is the first-step type; otherwise, the trend is set to "start" and the action type is the last-step type.

2) MLLM Design: To obtain a structured representation, we employ a multimodal large language model (MLLM) and formulate the problem as a text-plus-image-to-text transformation. As shown in Fig. 1, we prompt the MLLM, through in-context learning, to translate the complete scenario text, initial agent poses, and initial scene image into a YAML-like structured description. To improve output quality, we use a chain-of-thought prompting strategy: the MLLM first summarizes the scenario in a short sentence and then plans, agent by agent, how to generate the agent code c_a. The complete prompts are provided in Supp. B.

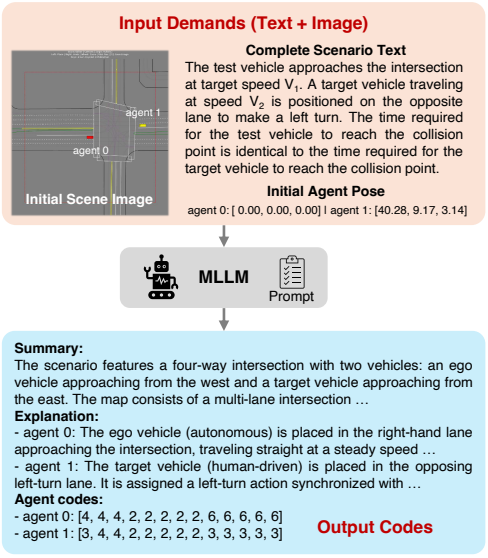


Fig. 1. Example of Interpreter's input and output.

B. Transformer-based Generator

The generator $S = \text{Generator}(p_0, c_a, m)$ takes the initial agent poses p_0, the agent codes c_a, and the preprocessed map m as input, and synthesizes a driving scenario S consistent with the user demand. The architecture of the generator is shown in Fig. 2.

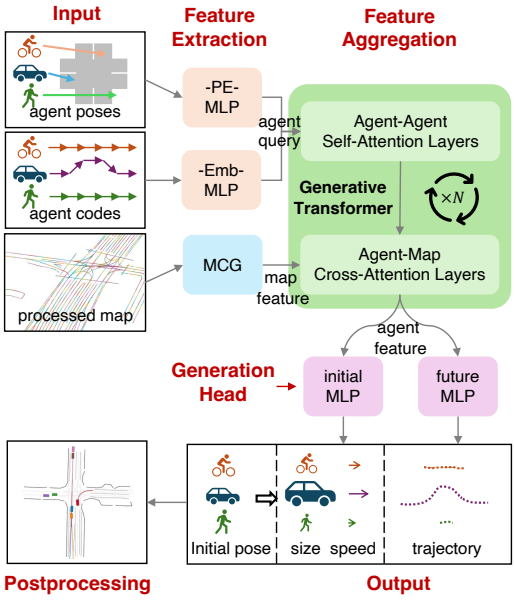


Fig. 2. The Architecture of Transformer-based Generator

1) Feature Extraction: The preprocessed map $m \in \mathbb{R}^{N_m \times A_m}$ contains N_m map points with A_m-dimensional attributes. To model complex spatial dependencies and contextual interactions among map points, we adopt multi-context gating blocks (MCG blocks) for feature fusion. MCG adaptively aggregates neighborhood information under different receptive fields, thereby capturing the relationship between local details and global structure and producing context-rich map features $E_m = \mathrm{MCG}_m(m) \in \mathbb{R}^{N_m \times D_m}$.

The initial agent pose $p_0 = (x_0, y_0, \theta_0) \in \mathbb{R}^{N_a \times 3}$ includes the position and heading at the first time step. Since such low-dimensional signals are difficult for an MLP to use to learn high-frequency variations directly, we first map them explicitly into a high-dimensional space using a sinusoidal positional encoding (PE) layer and then feed them into an MLP to extract pose features $E_p = \mathrm{MLP}_p(\mathrm{PE}(p_0)) \in \mathbb{R}^{N_a \times D_p}$. The agent code $c_a \in \mathbb{R}^{N_a \times A_a}$ contains discrete category, attribute, and action information. To convert it into a dense representation suitable for network learning, an embedding layer first maps the discrete attributes to continuous vectors, and an MLP further extracts high-level semantics $E_c = \mathrm{MLP}_c(\mathrm{Emb}(c_a)) \in \mathbb{R}^{N_a \times D_c}$, yielding the code feature $E_c \in \mathbb{R}^{N_a \times D_c}$. Finally, the pose feature and code feature are concatenated to form an agent feature $E_a = [E_p, E_c] \in \mathbb{R}^{N_a \times (D_p + D_c)}$ that integrates spatial geometry and semantic attributes.

2) Feature Aggregation: To better model interactions among agents, especially agent-map interactions, we feed the map features E_m and agent features E_a into Transformer layers. Each layer follows $E_a^{n+1} = \mathrm{AACA}(\mathrm{AMSA}(E_a^n), E_m), n = 1, \ldots, N$, where AACA and AMSA denote agent-map multi-head cross-attention and agent-agent multi-head self-attention, respectively. The output of each layer is used as the query for cross-attention with E_m. The output E_a^{n+1} of the last layer is the fused agent feature.

3) Generation Head: Based on the fused agent features, a simple MLP generates the initial and future states of each agent. Following TrafficGen [1], we predict the initial agent state, including the angle between heading and velocity direction, speed, and size. The initial state distribution is modeled with a Gaussian mixture model (GMM). For each attribute of agent i, the M-dimensional GMM parameters are predicted as $[\mu_i, \Sigma_i, \pi_i] = \mathrm{MLP}_{\mathrm{init}}(E_a^{n+1})$, where μ_i, Σ_i, and π_i denote the mean, covariance, and mixture weight of the M-dimensional GMM, respectively. We further predict the poses for the next $T-1$ steps and output K candidate future poses $\{p_{i,k,2:T}, P_{i,k}\}_{k=1}^{K} = \mathrm{MLP}(E_a^{n+1})$ for each agent, where $p_{i,k,2:T}$ denotes the pose at future step $T-1$ for agent i and mode k, and $P_{i,k}$ denotes its probability.

C. Processing, Training and Inference

1) Training Sample and Ground-Truth Generation: Ideally, the complete scenario text $\mathcal{T}$ would be part of each training sample and the real trajectories and related states would serve as ground truth. However, extracting or annotating paired scenario text from scenario datasets is prohibitively expensive, so it cannot be used as supervised input with real labels. Therefore, we predefine a rule set $[p_0, c_a] = \psi(S_a^*)$ that takes the reconstructed agent tracks S_a^* to extract the initial agent poses p_0 and agent codes c_a, and, by processing $m^* = \phi(S_m)$ from the map tracks S_m, obtains the preprocessed map m. The tuple (p_0, c_a, m) is used as the training sample, reducing the task to training the Generator. During training,

the reconstructed agent tracks S_a^* provide the scenario ground truth $s = [A, p]$, including the initial-attribute ground truth A and the future-pose ground truth p. The model $\hat{s} = \text{Generator}(p_0^*, c_a^*, m^*)$ produces the scenario output $\hat{s}$, and the Generator is trained by minimizing the discrepancy between s and $\hat{s}$.

2) Training Loss Design: We define the loss as two terms, the initial-scene loss $\mathcal{L}_{\text{initial}}$ and the future-scene loss $\mathcal{L}_{\text{future}}$:

$$\mathcal{L}(s, \hat{s}) = w_1 \mathcal{L}_{\text{initial}} + w_2 \mathcal{L}_{\text{future}}$$

For $\mathcal{L}_{\text{initial}}$, following TrafficGen [1], we use the negative log-likelihood loss computed from the predicted Gaussian mixture model of the attribute ground truth. For agent i and its attribute value $A_i = [i_i, v_i, s_i]$, including the angle i_i between heading and velocity direction, speed v_i, and size s_i, we use an MLP to predict the GMM mean μ_i, covariance Σ_i, and mixture weight π_i parameters. The constructed GMM is used to compute the likelihood of the attribute ground truth, and the complete initial-scene loss is expressed as $\mathcal{L}_{\text{initial}} = -\sum_{i=1}^{I} \log \text{GMM}_i(A_i)$. Here, GMM_i denotes the likelihood function of the predicted GMM initial-state parameters for agent i. These likelihoods are computed from the predicted GMM parameters.

For $\mathcal{L}_{\text{future}}$, following MultiPath [2], we adopt the mean squared error (MSE) loss and compute the predicted trajectory closest to the ground truth. For each agent i, we predict K' future poses and their probabilities $\{p_{i,k,2:T}, P_{i,k}\}_{k=1}^{K'} = \text{MLP}(F_i)$. Given the future-pose ground truth $p_{i,*,2:T}$, we compute, among the K' predictions, the index $k^* = \arg\min_k \sum_{t=2}^{T} \left(p_{i,k^*,t} - p_{i,*,t}\right)^2$ of the closest future pose. The complete future-scene loss is expressed as $\mathcal{L}_{\text{future}} = \sum_{i=1}^{N} \sum_{t=2}^{T} \left(p_{i,k^*,t} - p_{i,*,t}\right)^2$.

3) Inference Stage: For critical-scenario generation, based on the map description in the complete scenario text $\mathcal{T}$, the user provides the corresponding map code c_m. The map code is used to retrieve IDs of all matching scenarios from the preprocessed scenario dataset $\mathcal{D}_{\text{pre}}$, from which one driving scenario $\mathcal{S}$ is randomly sampled. The map tracks S_m in the scenario are rendered as an interactive image. According to the agent descriptions in the complete scenario text $\mathcal{T}$, the user places agents on the given map by mouse clicks and movements, forming the initial scene snapshot $\mathcal{I}$ and yielding the initial agent poses p_0. The interpreter $c_a^{\text{critical}} = \text{Interpreter}(\mathcal{T}, \mathcal{I}, p_0)$ outputs the critical agent codes c_a^{cri}, and the generator with post-reconstruction $\mathcal{S}^{\text{critical}} = \phi\left(p_0, c_a^{\text{critical}}, m\right)$ outputs the critical driving scenario.

Normal-scenario generation does not require the additional input $\mathcal{T}$, and is further simplified as follows: scenario IDs matching the requirements on agent density (low/medium/high) and road type (road segment/three-way intersection/four-way intersection/other) are retrieved from the preprocessed scenario dataset $\mathcal{D}_{\text{pre}}$, and one scenario is randomly sampled. From the driving scenario $\mathcal{S}$, we obtain the initial agent poses p_0, normal agent codes c_a^{normal}, and the preprocessed map m. The generator with post-reconstruction $\mathcal{S}^{\text{normal}} = \phi\left(p_0, c_a^{\text{normal}}, m\right)$ outputs the normal driving scenario.

The preprocessed scenario dataset $\mathcal{D}_{\text{pre}}$ consists of preprocessed scenarios $S = \{\text{id}: [c_a, c_m, m]\}$, each including agent codes c_a^*, a map code c_m, and a preprocessed map m. For the agent code $c_a \in \mathbb{Z}^{N_a \times 13}$, N_a denotes the number of agents. For each agent, dimension 1 encodes type (pedestrian/cyclist/human-driven vehicle/autonomous vehicle), dimension 2 encodes speed range (2.5 m/s interval), dimension 3 encodes size range (1 m interval), dimensions 4-8 encode the agent state for every 10 time steps

(start/keep/end), and dimensions 9-13 encode the agent action for every 10 time steps (stop/U-turn/left turn/left lane change/deceleration/constant speed/acceleration/right lane change/right turn). The map code $c_m \in \mathbb{Z}^{14}$ describes map information around the ego vehicle. Dimension 1 encodes road type (road segment/three-way intersection/four-way intersection/other), dimension 2 encodes the number of lanes (N), dimension 3 encodes the ego-lane ID (1/2/.../N), dimension 4 encodes the bicycle-lane ID (1/2/.../N/None), dimensions 5-9 encode traffic-light states for every 10 time steps (start, keep, end), and dimensions 10-14 encode traffic-light colors for every 10 time steps (red/yellow/green/flashing). For the preprocessed map $m \in \mathbb{R}^{N_m \times 6}$, N_m denotes the number of map elements; each map element includes a type (lane centerline, road edge, lane line, or other), a start position, an end position, and a traffic-light color.

III. EXPERIMENTS

1) Datasets: We use the publicly available Waymo Open Motion Dataset [3], which is collected from real-world driving. It contains approximately 67 K training scenarios and 3.5 K test scenarios.

2) Implementation: In the generator, the maximum numbers of map elements per scenario are set to 384 lane-centerline segments, 168 road-edge segments, 192 lane-line segments, and 32 other segments, and the maximum number of agents is 96. Scenarios are generated at 10 Hz for 50 time steps (5 s in total). Both the agent feature extractor and the generation head use two-layer MLPs. Map feature extraction uses five MCG blocks, and feature aggregation is implemented by a two-layer, four-head Transformer. *Each agent predicts* $K' = 12$ candidate future states. The weights of the initial-scene loss and future-scene loss are set to $w_1 = 1$ and $w_2 = 10$, respectively. In the overall Demand2Scenario framework, only Generator is trained, while Reconstructor and Interpreter are frozen. Training uses the AdamW optimizer for 100 epochs with a learning rate of 3×10^{-4} and a batch size of 256. A full Generator training run takes approximately 3 hours on eight NVIDIA A800 GPUs.

REFERENCES

[1] L. Feng, Q. Li, Z. Peng, S. Tan, and B. Zhou, "TrafficGen: Learning to generate diverse and realistic traffic scenarios," in *Proc. IEEE Int. Conf. Robot. Autom.*, Jul. 2023, pp. 3567–3575.

[2] Y. Chai, B. Sapp, M. Bansal, and D. Anguelov, "MultiPath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction," in *Proc. Conf. Robot Learn.*, May 2020, pp. 86–99.

[3] S. Ettinger et al., "Large scale interactive motion forecasting for autonomous driving: The Waymo Open Motion Dataset," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2021, pp. 9710–9719.

Supp. A Agent Code Extraction

Rather than directly annotating the raw data, we first annotate the raw dataset frame by frame; details are provided in the appendix. The priority of each action is then defined according to its rarity in the real world, with rarer actions assigned higher priority. The priority order is: U-turn > left/right lane change > left/right turn > acceleration/deceleration > constant speed > stop.

To annotate agent actions at each time step, we first determine whether each agent is stopped at time step t. An agent is considered stopped if its speed is below $v_{\min}$, with thresholds of 0.2 m/s for vehicles and cyclists and 0.1 m/s for pedestrians. Among non-stationary time steps, lane-change motions can be confused with straight driving and turning because their magnitude lies between the two; therefore, we extract lane-change time steps first. We use the lateral lane-offset curve of each vehicle to determine the start and end time steps of a lane change, while cyclists and pedestrians are assumed to have no lane-change behavior. For non-stationary and non-lane-change time steps, an agent is regarded as making a U-turn if its heading-angle change satisfies $h_{t+30} - h_{t-10} \geq 160$ degrees. On this basis, the agent is regarded as making a left turn if its heading-angle change satisfies $h_{t+10} - h_{t-10} \geq 15$ degrees, and a right turn if it satisfies $h_{t+10} - h_{t-10} \leq -15$ degrees; pedestrians are assumed to have no turning behavior. Finally, for time steps that are non-stationary, non-lane-change, and non-turning, we regard the motion as straight driving, including acceleration, deceleration, and constant speed. The agent is considered decelerating if its acceleration is below $a_{\min}$, accelerating if it is above $a_{\max}$, and moving at constant speed if it lies between $a_{\min}$ and $a_{\max}$. The acceleration threshold is set to $\pm 0.5\text{m/s}^2$ for vehicles and cyclists and $\pm 0.25\text{m/s}^2$ for pedestrians.

Supp. B MLLM Prompt Design

```
## Role Definition

You are a faithful format converter. Your task is to translate natural language traffic scenario descriptions, an initial scenario snapshot (image), and each agent's initial pose (position and heading) into a fixed-format **agent code**. You must ensure that the generated codes truthfully reflect the motion behaviors in the scenario and guarantee that collisions may occur between agents (if implied or required by the text description). The final output is only the list of agent codes.

## Expected Format

- **summary** (optional, for explanation): Briefly summarize the scenario, including the number of agents and how the potential collision point and initial speeds are determined based on the full scenario text, initial scenario snapshot, and agent initial poses.
- **explanation** (optional, for explanation): Explain why each agent (group) is placed in the scenario and how its actions fulfill the description requirements.
- **agent codes** (mandatory): A list of thirteen numerical values describing the attributes of each agent. Output in order: **agent 0 must be the ego car (autonomous vehicle)**. The remaining agents are sorted by increasing Euclidean distance from the ego car (according to the positions in `Agent Initial Poses`). Output format as follows (each line starts with `-- ` to clearly identify agent code lines in plain white background):
-- agent 0: [ , , , , , , , , , , , , ]
-- agent 1: [ , , , , , , , , , , , , ]
...
-- agent N-1: [ , , , , , , , , , , , , ]

text

> Note:
> - Do not output any map-related descriptions or codes. Each vector strictly contains 13 dimensions, values separated by commas.
> - The `-- ` prefix is required to distinguish agent code lines from ordinary text when displayed on a white background (non-code-block).

## Agent Code Attribute Meanings
```

Each agent code contains 13 dimensions, divided into initialization attributes and future action attributes.

Initialization Attributes (dimensions 0-2)

Dim	Name	Range	Meaning
0	Type	[1,4]	1 - pedestrian, 2 - cyclist, 3 - human-driven vehicle, 4 - autonomous vehicle (ego car)
1	Speed range index	[0,8]	Speed range, each interval 2.5 m/s. Actual speed v → index = floor(v/2.5). e.g., 21.2 m/s → index 8. **This value must be determined based on potential collision point calculation** (see rules below).
2	Size range index	[0,15]	Length range, each interval 1 m. Actual length L → index = floor(L/1). e.g., 8.1 m → index 8.

Future Action Attributes (dimensions 3-12)

Divide the next 5 seconds into 5 one-second intervals. Each interval contains an **action trend** and an **action type**.

Dim Range	Name	Range	Meaning
3-7	Action trend (per second)	[0,3]	0 - unknown, 1 - start, 2 - keep, 3 - finish
8-12	Action type (per second)	[0,9]	0 - unknown, 1 - stop, 2 - U-turn, 3 - left turn, 4 - left lane change, 5 - decelerate, 6 - constant, 7 - accelerate, 8 - right lane change, 9 - right turn

Action type restrictions:
- Pedestrian (type 1): only {0,1,5,6,7}
- Cyclist (type 2): only {0,1,3,5,6,7,9}
- Vehicle/ego car (type 3/4): all {0-9}

Action trend explanation: Actions like left turn, right turn, U-turn, lane changes usually last more than 5 seconds, so the trend sequence is often `[2,2,2,2,2]` (keep throughout the prediction window). If an action starts or ends within the window, set the corresponding position to 1 or 3.

Input Data Explanation

You receive the following three inputs:

1. **Full Scenario Text**: Natural language description of the scenario (including actions, interactions, collisions, etc.).
2. **Agent Initial Poses**: Each agent's initial position (x,y) and heading (radians). **The ego car's initial position is fixed at (0,0) and heading fixed at 0 radians**. Coordinate system definition: 0 rad → right (east), 1.57 rad → up (north), 3.14 rad → left (west), -1.57 rad → down (south). Other agents' headings can be given freely.
3. **Initial Scenario Snapshot**: Initial scene image containing agents and roads. In the image:
    - Red rectangle = ego car (autonomous vehicle)
    - Yellow rectangle = human-driven vehicle
    - Green rectangle = cyclist
    - Blue rectangle = pedestrian
    - Small triangle inside rectangle indicates heading
    - The image also shows roads (lane lines, intersections, stop lines, etc.) used to determine whether a collision point lies on the road and action timing.

Agent Code Generation Rules

Rule 1: Agent Order and Type
- **agent 0** must be the ego car corresponding to the red rectangle (type 4), with initial position and heading fixed at (0,0,0).

- The remaining agents (agent 1, 2, ..., N-1) are sorted by **increasing Euclidean distance from the ego car** (based on initial positions in `Agent Initial Poses`). If distances are equal, order arbitrarily.
- Type (dimension 0) is determined by the rectangle color in the image: red→4, yellow→3, green→2, blue→1.

Rule 2: Determining Speed Range Index (dimension 1) – Collision Point Speed Calculation

To ensure that a potential collision (or interaction) described in the text can actually happen, the initial speeds of the ego car and other agents should be such that they **arrive at the potential collision point simultaneously**.

Steps:
1. **Identify potential collision point**: Combine `Full Scenario Text` (described actions such as go straight, turn left, lane change, etc.), `Agent Initial Poses` (initial positions and headings), and the road structure in `Initial Scenario Snapshot` (intersections, lane lines, stop lines, etc.) to infer the most likely conflict point that lies on the road. Examples:
   - Intersection: two vehicles from different directions, collision point at the center of the intersection (must be within drivable road area).
   - Lane change: rear vehicle and front vehicle, collision point at the rear-side of the front vehicle (must be within same-direction lane).
   - Vehicle and pedestrian: collision point at the intersection of pedestrian crossing path and vehicle trajectory (typically at a crosswalk).
2. **Calculate distance from each agent to collision point**: Using initial position and heading, measure path length along the expected trajectory (inferred from action type). Ignore lateral offset; approximate as straight-line distance (if trajectory is straight) or arc length.
3. **Set simultaneous arrival condition**: If agents A and B need to arrive at the collision point at the same time (or within a safety threshold), then `distance_A / speed_A = distance_B / speed_B = T` (arrival time). Typically fix the ego car's arrival time T (e.g., 3–5 seconds), then compute other agents' speeds: `speed_B = distance_B / T`.
4. **Convert to speed range index**: For computed actual speed v (m/s), take `index = min(8, floor(v / 2.5))`. If v > 20 m/s, index = 8. If v < 0, index = 0 (should not happen normally).
5. **No clear collision point**: If the text does not imply any collision or interaction, use reasonable speeds based on scenario type (e.g., 10–15 m/s for urban roads, 25–30 m/s for highways) and take the corresponding index.

> **Note**: This rule also applies to non-ego agents. If an agent does not collide with the ego but interacts with another agent, compute speeds based on that interaction pair. The collision point must lie within the drivable road area shown in the `Initial Scenario Snapshot`.

Rule 3: Size Range Index (dimension 2)
- Determine length based on vehicle type described in `Full Scenario Text` (e.g., car, SUV, truck, bicycle, pedestrian). If not specified, use typical values:
  - Pedestrian: 0.5 m → index 0
  - Cyclist: 1.8 m → index 1
  - Car: 4.5 m → index 4
  - SUV: 5.0 m → index 5
  - Truck: 12 m → index 12
- Index = floor(length / 1), capped at 15.

Rule 4: Action Trend (dimensions 3-7) and Action Type (dimensions 8-12)
- Extract each agent's action sequence for the next 5 seconds from `Full Scenario Text` and `Agent Initial Poses`. Road elements in `Initial Scenario Snapshot` (e.g., stop lines, intersections) affect action timing. Note: these road elements are only used for action timing judgment; no map description is required in output.
- **Action type sequence generation**:
  - If a described action (e.g., "turn left") completes within the 5-second window, the sequence might be `[3,3,3,3,3]` (continuous left turn) or with start/end markers. Typically actions like left/right turn, U-turn, lane change do not finish within 5 seconds, so they are set as constant actions (type 2/3/4/8/9) with trend

2.
  - Decelerate/accelerate/constant: based on speed change description. Example: "decelerate when approaching stop line" → first 1 second constant, next 4 seconds decelerate.
  - Stop: if the text says "stop before stop line", action type sequence might be `[6,5,5,1,1]` (constant → decelerate → stop).
- **Action trend sequence**: one-to-one with action types, indicating the state of that action at each time point:
  - If action is ongoing from the start and continues to the end: `[2,2,2,2,2]`
  - If action starts in the middle: e.g., start turning left at second 2 → `[2,1,2,2,2]` (keep → start → keep → keep → keep)
  - If action ends in the middle: e.g., end deceleration at second 4 → `[2,2,2,3,2]` (keep → keep → keep → finish → keep)

Rule 5: Coordinate System Explanation
- All heading angles are in radians, defined as:
  - 0 rad: right (east)
  - 1.57 rad (≈π/2): up (north)
  - 3.14 rad (≈π): left (west)
  - -1.57 rad (≈-π/2): down (south)
- When calculating distances and trajectories, infer the path based on heading and action type.

Comprehensive Example (Correct Geometry: ego facing right, lower-right vehicle facing up)

Input:
- **Full Scenario Text**: `"The ego car goes straight through an intersection. A human-driven vehicle from its lower right approaches vertically (heading up). The two vehicles will collide at the center of the intersection."`
- **Agent Initial Poses**: `ego: (0,0,0); lower-right vehicle: (30, -10, 1.57)`
  Explanation: ego at (0,0) heading right (0 rad); lower-right vehicle at (30,-10) heading up (1.57 rad). Note: 1.57 rad heading up means this vehicle travels vertically upward from below.
- **Initial Scenario Snapshot**: Red rectangle (ego) at (0,0) heading right; yellow rectangle at (30,-10) heading up; the image shows a crossroads with roads along the x and y axes, intersection center at (30,0) (drivable area).

Reasoning:
1. **Potential collision point**: Combining the full scenario text ("collide at the center of the intersection") and the road structure in the snapshot (crossroads), both vehicles travel straight. Ego trajectory: from (0,0) right → line y=0. Lower-right vehicle trajectory: from (30,-10) up → line x=30. Intersection point is (30,0), which lies at the intersection center in the snapshot (drivable area) – valid.
2. **Calculate distances**:
   - Ego to collision point: from (0,0) to (30,0) straight distance = 30 m.
   - Lower-right vehicle to collision point: from (30,-10) to (30,0) straight distance = 10 m.
3. **Simultaneous arrival condition**: Set arrival time T = 2 seconds.
   - Ego speed = 30 m / 2 s = 15 m/s → speed index = floor(15/2.5) = 6.
   - Lower-right vehicle speed = 10 m / 2 s = 5 m/s → speed index = floor(5/2.5) = 2.
4. **Size**: Not specified, use typical car length 4.5 m → index 4.
5. **Actions**: Both vehicles go straight, maintain constant speed for the next 5 seconds → action type 6 (constant), trend 2 (keep) for all 5 seconds.
6. **Order**: Distance from lower-right vehicle to ego = sqrt((30-0)^2 + (-10-0)^2) = sqrt(900+100) = sqrt(1000) ≈ 31.62 m. Only two agents, agent 0 = ego, agent 1 = that vehicle.

Output:
-- agent 0: [4, 6, 4, 2, 2, 2, 2, 2, 6, 6, 6, 6, 6]
-- agent 1: [3, 2, 4, 2, 2, 2, 2, 2, 6, 6, 6, 6, 6]

text

Explanation: In this example, ego is fixed heading right (0 rad), the other vehicle heading up (1.57 rad).

Their trajectories intersect orthogonally at (30,0). Initial speeds are 15 m/s (index 6) and 5 m/s (index 2), ensuring simultaneous arrival at the collision point. The geometry is fully correct, and the collision point lies on the road as shown in the initial scenario snapshot. The `-- ` prefix in the output clearly identifies each agent code line on a white background (non-code-block).

Please generate the agent codes according to the above rules for the given inputs (Full Scenario Text, Agent Initial Poses, Initial Scenario Snapshot).